

Secure Code Review Assignment – Instructions

Objective

You have been provided with the source code for **Scaler Support Desk**, a deliberately vulnerable web application.

The objective of this assignment is to perform a **Secure Code Review**, identify security issues in the source code, and distinguish between **True Positives** and **False Positives**.

The primary focus of this assignment is **code analysis**, **vulnerability validation**, and **security reasoning**, rather than exploitation.

Assignment Requirements

Review the entire source code and identify potential security findings (Unzip the file supportdesk-app.zip and review the code)

You may also use Static Application Security Testing (SAST) tools to assist your analysis.

Recommended Tool:

- Semgrep (Recommended)

You may also use:

- SonarQube
- Checkmarx
- CodeQL
- Bandit
- Any other SAST tool of your choice

Note: Tool output should be treated as a starting point for analysis, not as the final conclusion. Every finding must be manually reviewed and verified before classifying it as a True Positive or False Positive.

Optional Dynamic Verification

Students who wish to further validate their findings may optionally run the application locally (by running app.py file) and reproduce the vulnerabilities.

The evaluation will primarily be based on the quality of your secure code review and vulnerability analysis.

Submission Requirements

For every finding identified, include the following information in your report.

1. Vulnerability Title

Example:

- SQL Injection
 - Command Injection
 - Cross-Site Scripting
 - Insecure File Upload
-

2. Classification

Classify every finding as either:

- True Positive
- False Positive

Provide justification for your classification.

3. Source File Information

Mention:

- File Name
- Function Name (if applicable)
- Vulnerable Line Number(s)

Example:

File:

app/routes/tickets.py

Line:

142–150

4. Screenshot of SAST Tool Output

Include a screenshot showing the finding generated by the static analysis tool.

Examples:

- Semgrep Result
 - SonarQube Finding
 - Checkmarx Finding
-

5. Screenshot of Vulnerable Code

Take a screenshot of the vulnerable source code.

Clearly highlight:

- The vulnerable line(s)
- The affected function or code block

The highlighted code should be clearly visible and readable.

6. Why is it Vulnerable?

Explain in your own words:

- Why the code is vulnerable
- How the vulnerability could be exploited
- What makes it a True Positive or False Positive

Avoid copying explanations directly from the internet or AI tools.

7. Security Impact

Explain the potential impact if the vulnerability exists.

Examples:

- Remote Code Execution
- Authentication Bypass
- Information Disclosure
- Privilege Escalation
- Denial of Service

8. Recommended Remediation

Provide an appropriate remediation for every True Positive.

Examples:

- Input Validation
- Parameterized Queries
- Output Encoding
- Authentication Checks
- Secure File Validation
- Principle of Least Privilege

If the finding is classified as a False Positive, explain why remediation is not required.

Deliverable Format

Submit a single PDF document containing all findings.

For every finding, include:

- Vulnerability Title
 - Classification (True Positive / False Positive)
 - File Name
 - Vulnerable Line Number(s)
 - Screenshot of SAST Tool Result
 - Screenshot of Vulnerable Code (Highlighted)
 - Explanation
 - Security Impact
 - Recommended Remediation
-

Evaluation Criteria

Your submission will be evaluated based on:

- Accuracy of identifying vulnerabilities
- Correct classification of True Positives and False Positives
- Quality of code analysis
- Understanding of secure coding concepts
- Quality of explanations
- Correct identification of vulnerable line numbers

- Appropriate remediation recommendations
- Overall report quality and documentation

Students are encouraged to rely on their own analysis rather than solely depending on automated tool output. The goal of this assignment is to develop the ability to review code, validate security findings, and make informed security decisions.